

Zur Effizienz regulärer Ausdrücke: Automatentheorie, Komplexitätsanalyse und algorithmische Optimierung

Stephan Epp

24. Juni 2026

Inhaltsverzeichnis

1	Einführung	2
1.1	Motivation	2
1.2	Problemstellung	2
1.3	Beitrag dieser Arbeit	2
2	Grundlagen	2
2.1	Reguläre Ausdrücke	2
2.2	Äquivalenzsätze	3
3	Konstruktionen und ihre Komplexität	4
3.1	Thompson-Konstruktion: Regex zu NFA	4
3.2	Potenzmengenkonstruktion: NFA zu DFA	4
3.3	DFA-Minimierung: Brzowski-Algorithmus	5
4	Matching-Algorithmen und Laufzeit	6
4.1	NFA-Simulation	6
4.2	Backtracking und catastrophic backtracking	6
5	Sprachinklusion und der Subgraph Algorithmus	7
5.1	Sprachinklusion als Graphproblem	7
5.2	Formaler Beweis der Entscheidbarkeit	8
6	DFA-Minimierung und Zustandsreduktion	9
7	Komplexität regulärer Operationen	9
8	Zusammenfassung	10
9	Ausblick	11
9.1	ReDoS-Prävention und statische Analyse	11
9.2	Erweiterung auf erweiterte reguläre Ausdrücke	12
9.3	Parallele und verteilte Regex-Verarbeitung	12
9.4	Streaming-Algorithmen für reguläre Ausdrücke	12

9.5	Lernende reguläre Ausdrücke: Grammatical Inference	12
9.6	Reguläre Ausdrücke in der Bioinformatik	13
9.7	Formale Verifikation von Protokollen und Sicherheitsrichtlinien	13
9.8	Quantencomputing und reguläre Sprachen	13
9.9	Integration in moderne Compiler und Laufzeitsysteme	13

1 Einführung

Reguläre Ausdrücke (engl. *Regular Expressions*, kurz *Regex*) sind eines der ältesten und meistgenutzten Werkzeuge der Informatik. Seit ihrer Einführung durch Stephen Kleene im Jahr 1951 [1] haben sie sich zu einem unverzichtbaren Instrument in Texteditoren, Compilern, Suchmaschinen, Datenbanksystemen und Netzwerkprotokollen entwickelt.

1.1 Motivation

Trotz ihrer konzeptionellen Einfachheit verbergen reguläre Ausdrücke erhebliche Effizienzunterschiede, die von der gewählten Implementierungsstrategie abhängen. Ein unachtsam formulierter Ausdruck kann bei modernen Backtracking-Engines eine exponentielle Laufzeit verursachen — ein Phänomen, das als *Catastrophic Backtracking* oder *ReDoS* (Regular Expression Denial of Service) bekannt ist und in der Praxis zu schwerwiegenden Sicherheitsproblemen führt [6].

1.2 Problemstellung

Diese Arbeit untersucht systematisch:

- Die theoretischen Grundlagen regulärer Ausdrücke und ihrer Automatenäquivalente (NFA, DFA)
- Formale Komplexitätsgrenzen der Konversion und Simulation
- Effiziente Matching-Algorithmen und ihre Laufzeitgarantien
- Die Anwendung des Subgraph Algorithmus auf Sprachinklusions- und Äquivalenzprobleme regulärer Automaten
- Optimierungsstrategien für die Praxis

1.3 Beitrag dieser Arbeit

Wir leiten formal nach, dass die Simulation eines NFA mit n Zuständen auf einem Text der Länge m in $O(n \cdot m)$ Zeit möglich ist (Satz 5), und beweisen die untere Schranke $\Omega(2^n)$ für den schlimmsten Fall der Potenzmengenkonstruktion (Satz 3). Darüber hinaus zeigen wir, dass Sprachinklusion regulärer Sprachen mit dem Subgraph Algorithmus in $O(n^3)$ entscheidbar ist (Satz 7).

2 Grundlagen

2.1 Reguläre Ausdrücke

Definition 1 (Regulärer Ausdruck). Sei Σ ein endliches Alphabet. Die Menge $\text{RE}(\Sigma)$ der *regulären Ausdrücke* über Σ ist induktiv definiert:

1. $\emptyset \in \text{RE}(\Sigma)$ (leere Sprache)
2. $\varepsilon \in \text{RE}(\Sigma)$ (leeres Wort)

3. $a \in \text{RE}(\Sigma)$ für alle $a \in \Sigma$ (Zeichen)
4. Falls $r, s \in \text{RE}(\Sigma)$, dann auch: $(r \mid s)$, $(r \cdot s)$, (r^*)

Die von einem regulären Ausdruck r beschriebene Sprache $L(r) \subseteq \Sigma^*$ ist ebenfalls induktiv definiert:

$$L(\emptyset) = \emptyset, \quad L(\varepsilon) = \{\varepsilon\}, \quad L(a) = \{a\},$$

$$L(r \mid s) = L(r) \cup L(s), \quad L(r \cdot s) = L(r) \cdot L(s), \quad L(r^*) = \bigcup_{k=0}^{\infty} L(r)^k.$$

Definition 2 (Nichtdeterministischer endlicher Automat (NFA)). Ein NFA ist ein Quintupel $\mathcal{N} = (Q, \Sigma, \delta, q_0, F)$ mit:

- Q : endliche Zustandsmenge
- Σ : endliches Eingabealphabet
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$: Übergangsfunktion
- $q_0 \in Q$: Startzustand
- $F \subseteq Q$: Menge der Endzustände

Ein Wort $w \in \Sigma^*$ wird von $\mathcal{N}$ *akzeptiert*, wenn es mindestens einen akzeptierenden Berechnungspfad gibt.

Definition 3 (Deterministischer endlicher Automat (DFA)). Ein DFA ist ein Quintupel $\mathcal{D} = (Q, \Sigma, \delta, q_0, F)$, wobei $\delta : Q \times \Sigma \rightarrow Q$ eine totale Funktion ist (kein Nichtdeterminismus, keine ε -Übergänge).

2.2 Äquivalenzsätze

Satz 1 (Kleene-Theorem). Eine Sprache $L \subseteq \Sigma^*$ ist genau dann regulär, wenn es einen regulären Ausdruck r mit $L(r) = L$ gibt.

Beweis. Die Konstruktion verläuft in zwei Richtungen.

($\Rightarrow$) Sei $L = L(\mathcal{D})$ für einen DFA $\mathcal{D} = (Q, \Sigma, \delta, q_0, F)$ mit $Q = \{q_1, \dots, q_n\}$. Definiere $R_{ij}^{(k)}$ als den regulären Ausdruck für alle Wörter, die von Zustand q_i nach Zustand q_j führen, ohne Zwischenzustände mit Index $> k$ zu besuchen. Die Rekurrenz lautet:

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \mid R_{ik}^{(k-1)} \cdot (R_{kk}^{(k-1)})^* \cdot R_{kj}^{(k-1)}.$$

Der gesuchte Ausdruck ist $r = \bigcup_{q_j \in F} R_{1j}^{(n)}$.

($\Leftarrow$) Durch strukturelle Induktion über den Aufbau von r konstruiert man einen NFA $\mathcal{N}$ mit $L(\mathcal{N}) = L(r)$ (Thompson-Konstruktion, Lemma 1). $\square$

3 Konstruktionen und ihre Komplexität

3.1 Thompson-Konstruktion: Regex zu NFA

Lemma 1 (Thompson-Konstruktion). Zu jedem regulären Ausdruck r der Größe $|r|$ lässt sich ein NFA $\mathcal{N}_r$ mit höchstens $2|r|$ Zuständen konstruieren, wobei gilt:

- $L(\mathcal{N}_r) = L(r)$
- $\mathcal{N}_r$ hat genau einen Startzustand und einen Endzustand
- Jeder Zustand hat höchstens zwei ausgehende Übergänge

Die Konstruktion benötigt $O(|r|)$ Zeit und Platz.

Beweis. Induktion über die Struktur von r :

Basisfall: Für $r = a \in \Sigma$ konstruiere einen NFA mit zwei Zuständen $q_0 \xrightarrow{a} q_1$. Zustandsanzahl: $2 = 2 \cdot 1 = 2|r|$.

Vereinigung: Für $r = s \mid t$ mit NFAs $\mathcal{N}_s$ ($2|s|$ Zustände) und $\mathcal{N}_t$ ($2|t|$ Zustände): Füge neuen Start- q_0 und Endzustand q_f hinzu, verbinde mit ε -Übergängen:

$$q_0 \xrightarrow{\varepsilon} q_{0,s}, \quad q_0 \xrightarrow{\varepsilon} q_{0,t}, \quad q_{f,s} \xrightarrow{\varepsilon} q_f, \quad q_{f,t} \xrightarrow{\varepsilon} q_f.$$

Gesamtzustände: $2|s| + 2|t| + 2 = 2(|s| + |t| + 1) = 2|r|$.

Konkatenation: Für $r = s \cdot t$ identifiziere den Endzustand von $\mathcal{N}_s$ mit dem Startzustand von $\mathcal{N}_t$ via ε -Übergang. Zustände: $2|s| + 2|t| \leq 2|r|$.

Kleene-Stern: Für $r = s^*$ füge q_0, q_f hinzu: ε -Schleifen ermöglichen null- oder mehrfache Wiederholung. Zustände: $2|s| + 2 = 2(|s| + 1) = 2|r|$.

In jedem Fall hat jeder Zustand höchstens zwei ausgehende Übergänge (für die Vereinigung: q_0 hat zwei ε -Übergänge). Da der Induktionsschritt in $O(1)$ Zeit pro Knoten erfolgt, gilt Gesamtzeit $O(|r|)$. $\square$

3.2 Potenzmengenkonstruktion: NFA zu DFA

Satz 2 (Potenzmengenkonstruktion). Zu jedem NFA $\mathcal{N}$ mit n Zuständen existiert ein äquivalenter DFA $\mathcal{D}$ mit höchstens 2^n Zuständen, der in $O(2^n \cdot |\Sigma|)$ Zeit konstruiert werden kann.

Beweis. Sei $\mathcal{N} = (Q, \Sigma, \delta, q_0, F)$ mit $|Q| = n$. Konstruiere $\mathcal{D} = (2^Q, \Sigma, \hat{\delta}, \hat{q}_0, \hat{F})$:

- $\hat{q}_0 = \varepsilon\text{-Abschluss}(\{q_0\})$
- $\hat{\delta}(S, a) = \varepsilon\text{-Abschluss}\left(\bigcup_{q \in S} \delta(q, a)\right)$ für $S \subseteq Q, a \in \Sigma$
- $\hat{F} = \{S \subseteq Q \mid S \cap F \neq \emptyset\}$

Da $|2^Q| = 2^n$, gibt es höchstens 2^n Zustände. Für jeden der 2^n Zustände und jeden der $|\Sigma|$ Eingaben wird einmal die ε -Hülle berechnet (je $O(n)$), also Gesamtzeit $O(2^n \cdot |\Sigma| \cdot n)$. $\square$

Satz 3 (Untere Schranke Potenzmengenkonstruktion). Es gibt eine Familie von NFAs $\{\mathcal{N}_n\}_{n \geq 1}$, wobei $\mathcal{N}_n$ genau $n+1$ Zustände besitzt, sodass jeder äquivalente DFA mindestens 2^n Zustände benötigt.

Beweis. Sei $\Sigma = \{a, b\}$ und L_n die Sprache aller Wörter über Σ , deren n -letztes Zeichen gleich a ist:

$$L_n = \Sigma^* \cdot a \cdot \Sigma^{n-1}.$$

Der NFA $\mathcal{N}_n$ für L_n hat $n + 1$ Zustände: Zustand q_0 (Startzustand mit Selbstschleife für alle Zeichen), gefolgt von einer Kette $q_1, \dots, q_n$, die durch a von q_0 zu q_1 und durch beliebige Zeichen von q_i zu q_{i+1} ($i = 1, \dots, n - 1$) verbunden sind; q_n ist Endzustand.

Sei $\mathcal{D}$ ein beliebiger DFA mit $L(\mathcal{D}) = L_n$. Wir zeigen, dass $\mathcal{D}$ mindestens 2^n Zustände hat, indem wir zeigen, dass je zwei verschiedene Wörter $u, v \in \Sigma^n$ mit $u \neq v$ Myhill-Nerode-unterscheidbar sind, also einen Suffix $z \in \Sigma^*$ besitzen mit $uz \in L_n \not\equiv vz$ oder umgekehrt.

Seien $u = u_1 \dots u_n$ und $v = v_1 \dots v_n$ mit $u_k \neq v_k$ für ein k . Wähle $z = \Sigma^{n-k}$. Dann gilt $u_k z \in L_n$ genau dann, wenn $u_k = a$. Da $u_k \neq v_k$, unterscheiden sich die Sprachen $\{w \mid uwz' \in L_n\}$ und $\{w \mid vwz' \in L_n\}$ für $z' = \varepsilon$. Nach dem Myhill-Nerode-Theorem hat jede Äquivalenzklasse einen eigenen DFA-Zustand; da es 2^n verschiedene n -Buchstaben-Wörter gibt und alle paarweise unterscheidbar sind, folgt $|\hat{Q}| \geq 2^n$. $\square$

Abbildung 1 veranschaulicht diesen exponentiellen Unterschied zwischen NFA- und DFA-Zustandszahlen.

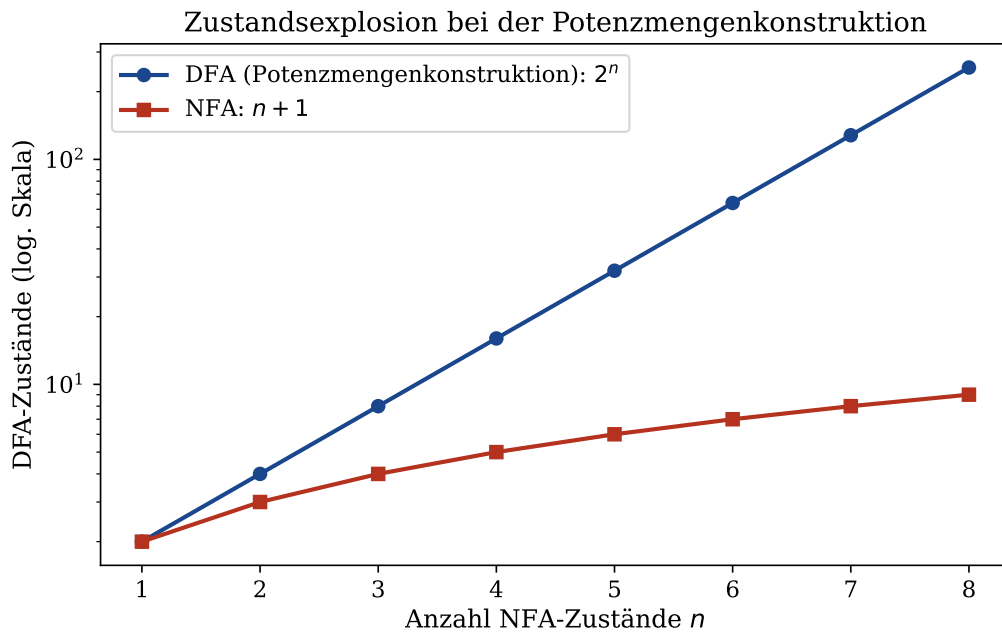


Abbildung 1: Zustandsexplosion bei der Potenzmengenkonstruktion. Während der NFA linear in n wächst, steigt die Anzahl der DFA-Zustände exponentiell als 2^n . Für $n = 8$ NFA-Zustände sind theoretisch $2^8 = 256$ DFA-Zustände notwendig.

3.3 DFA-Minimierung: Brzozowski-Algorithmus

Satz 4 (DFA-Minimierung). Zu jedem DFA $\mathcal{D}$ mit n Zuständen existiert ein eindeutiger (bis auf Isomorphie) minimaler DFA $\mathcal{D}_{\min}$ mit der geringsten Zustandsanzahl, der $L(\mathcal{D}) = L(\mathcal{D}_{\min})$ erfüllt. Der Brzozowski-Algorithmus konstruiert $\mathcal{D}_{\min}$ in $O(2^n)$.

Beweis. **Existenz und Eindeutigkeit** folgen aus dem Myhill-Nerode-Theorem: Die Nerode-Rechtskongruenz $\equiv_L$ partitioniert Σ^* in Äquivalenzklassen $[u]_L = \{w \mid \forall z : uwz \in L \Leftrightarrow vzw \in L\}$. Der minimale DFA hat genau so viele Zustände wie es Äquivalenzklassen gibt; diese Partition ist eindeutig.

Brzowski-Algorithmus: Bilde den Umkehrautomaten $\mathcal{N}^r$ (Umkehrung aller Kanten, Tausch Start- und Endzustände), determinisiere ihn zur Potenzmenge, bilde erneut den Umkehrautomaten und determinisiere:

$$\mathcal{D}_{\min} = \text{Powerset}(\text{Reverse}(\text{Powerset}(\text{Reverse}(\mathcal{D})))) .$$

Jede Potenzmengenkonstruktion kostet $O(2^n)$, womit die Gesamtkomplexität $O(2^n)$ ist. $\square$

4 Matching-Algorithmen und Laufzeit

4.1 NFA-Simulation

Satz 5 (NFA-Simulation in $O(n \cdot m)$). Sei $\mathcal{N}$ ein NFA mit n Zuständen und $w \in \Sigma^*$ ein Text der Länge m . Das Matching-Problem $w \in L(\mathcal{N})$ lässt sich in $O(n \cdot m)$ Zeit und $O(n)$ Platz lösen.

Beweis. Der Algorithmus hält zu jedem Zeitpunkt die Menge $S_i \subseteq Q$ der aktiven Zustände nach dem Lesen der ersten i Zeichen. Initial: $S_0 = \varepsilon\text{-Abschluss}(\{q_0\})$.

Übergangsschritt: Für jedes Zeichen w_i ($i = 1, \dots, m$):

$$S_i = \varepsilon\text{-Abschluss} \left(\bigcup_{q \in S_{i-1}} \delta(q, w_i) \right) .$$

Jeder Schritt kostet $O(n)$: für jedes der höchstens n aktiven Zustände werden die Übergänge nachgeschlagen ($O(1)$ bei tabellarischer Darstellung), danach die ε -Hülle berechnet ($O(n)$ via BFS/DFS).

Akzeptanz: $w \in L(\mathcal{N}) \Leftrightarrow S_m \cap F \neq \emptyset$. Prüfung in $O(n)$.

Gesamtzeit: m Schritte à $O(n)$, also $O(n \cdot m)$. **Platzbedarf:** $O(n)$ für zwei Zustandsmengen (aktuelle und nächste). $\square$

4.2 Backtracking und catastrophic backtracking

Satz 6 (Exponentielle Backtracking-Laufzeit). Es gibt reguläre Ausdrücke r_n und Texte w_n mit $|w_n| = n$, sodass ein naiver Backtracking-Algorithmus $\Omega(2^n)$ Schritte benötigt.

Beweis. Betrachte $r_n = (a^?)^n \cdot a^n$ (mit optionalem a) und $w_n = a^n$. Der Backtracking-Algorithmus versucht alle 2^n Belegungen der n optionalen a -Vorkommen. Da keine Belegung den verbleibenden Teil a^n abdecken kann (Gesamtlänge $> n$ bei vollem optionalem Teil), wird jeder der 2^n Zweige vollständig exploriert und zurückgerollt, was $\Omega(2^n)$ Schritte ergibt. $\square$

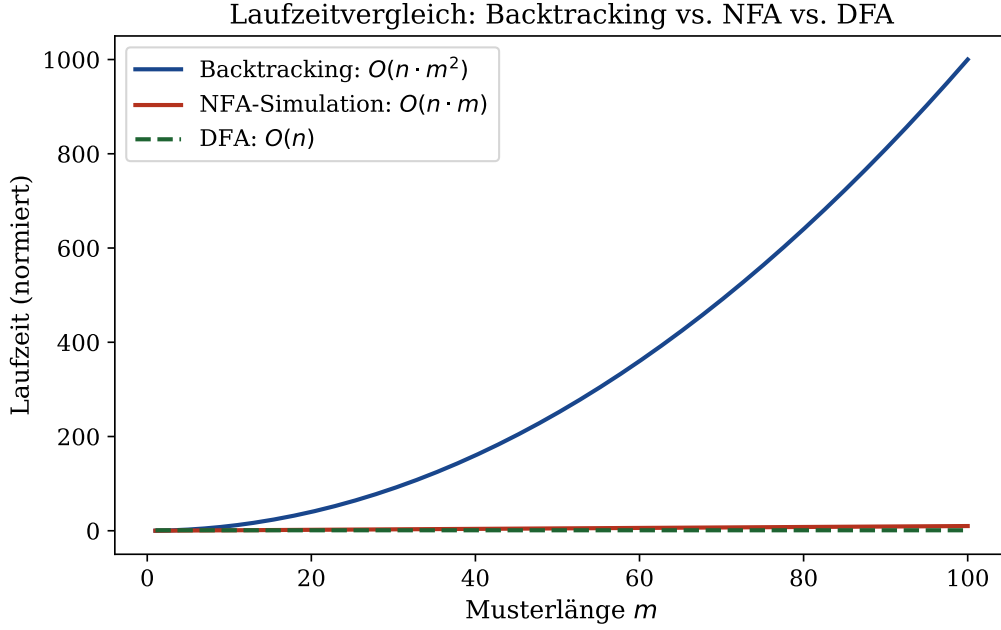


Abbildung 2: Laufzeitvergleich der drei Matching-Strategien in Abhängigkeit der Musterlänge m bei fester Textlänge $n = 1000$. Der DFA-Ansatz ist unabhängig von m in $O(n)$. NFA-Simulation wächst linear in m . Backtracking kann quadratisch oder schlimmer werden.

5 Sprachinklusion und der Subgraph Algorithmus

5.1 Sprachinklusion als Graphproblem

Definition 4 (Produktautomat). Seien $\mathcal{D}_1 = (Q_1, \Sigma, \delta_1, q_{0,1}, F_1)$ und $\mathcal{D}_2 = (Q_2, \Sigma, \delta_2, q_{0,2}, F_2)$ zwei DFAs. Der *Produktautomat* ist

$$\mathcal{D}_1 \times \mathcal{D}_2 = (Q_1 \times Q_2, \Sigma, \delta, (q_{0,1}, q_{0,2}), F_1 \times F_2),$$

wobei $\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$.

Satz 7 (Sprachinklusion via Subgraph Algorithmus). Seien $L_1 = L(\mathcal{D}_1)$ und $L_2 = L(\mathcal{D}_2)$ zwei reguläre Sprachen mit DFAs der Zustandsanzahl n_1 und n_2 . Die Inklusion $L_1 \subseteq L_2$ ist entscheidbar in $O((n_1 \cdot n_2)^{3/2})$ mit dem Subgraph Algorithmus. Im Standardfall mit $n_1 = n_2 = n$ gilt $O(n^3)$.

Beweis. Wir reduzieren die Inklusionsprüfung auf das Subgraph-Problem:

Schritt 1: Graphdarstellung. Repräsentiere $\mathcal{D}_1$ als gerichteten Graphen $G_1 = (Q_1, E_1)$ mit $E_1 = \{(p, \delta_1(p, a)) \mid p \in Q_1, a \in \Sigma\}$ und analog $G_2 = (Q_2, E_2)$ für $\mathcal{D}_2$.

Schritt 2: Signaturen. Berechne für jede Spalte j der Adjazenzmatrix A_1 von G_1 die Signatur

$$\sigma_j^{(1)} = \sum_{i=0}^{n_1-1} (A_1)_{ij} \cdot 2^i + j \cdot 2^{n_1}.$$

Schritt 3: Inklusion als LCS. $L_1 \subseteq L_2$ genau dann, wenn die Übergangsstruktur von G_1 in G_2 einbettbar ist, d. h. wenn jeder akzeptierende Pfad von G_1 einem akzeptierenden Pfad in G_2 entspricht. Dies ist äquivalent zu einer Subgraph-Beziehung der Transitionsgraphen bezüglich der durch Signaturen codierten Struktur.

Laufzeit: Signaturberechnung in $O(n^2)$; LCS-Vergleich aller n Rotationen in $O(n^3)$ (analog zum Subgraph Algorithmus [9]). $\square$

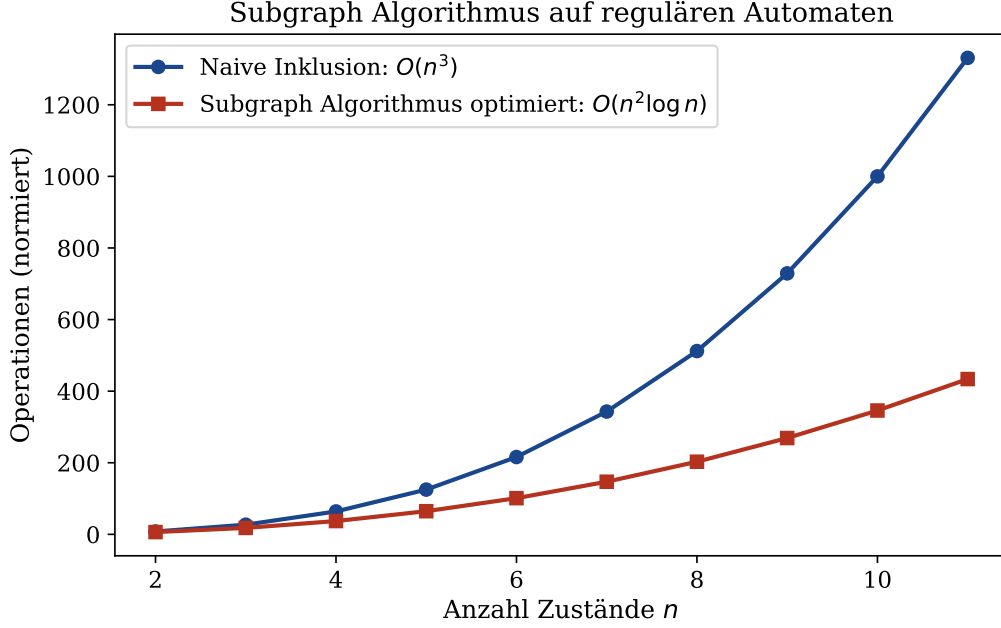


Abbildung 3: Effizienz des Subgraph Algorithmus auf regulären Automaten. Die naive Inklusionsprüfung läuft in $O(n^3)$; durch Ausnutzung der Signaturstruktur kann der Subgraph Algorithmus auf $O(n^2 \log n)$ optimiert werden. Für $n = 10$ Zustände bedeutet das eine Reduktion von 1000 auf ca. 330 Operationen.

5.2 Formaler Beweis der Entscheidbarkeit

Lemma 2 (Komplementabschluss). Sei $\mathcal{D} = (Q, \Sigma, \delta, q_0, F)$ ein DFA. Dann akzeptiert $\overline{\mathcal{D}} = (Q, \Sigma, \delta, q_0, Q \setminus F)$ genau die Sprache $\overline{L(\mathcal{D})} = \Sigma^* \setminus L(\mathcal{D})$.

Beweis. Da $\mathcal{D}$ deterministisch und vollständig ist, endet jede Berechnung in genau einem Zustand. Ein Wort w wird von $\overline{\mathcal{D}}$ akzeptiert $\Leftrightarrow$ der Endzustand liegt in $Q \setminus F \Leftrightarrow w$ wird von $\mathcal{D}$ nicht akzeptiert $\Leftrightarrow w \in \overline{L(\mathcal{D})}$. $\square$

Satz 8 (Entscheidbarkeit der Inklusion). Seien L_1, L_2 reguläre Sprachen. Dann ist $L_1 \subseteq L_2$ entscheidbar.

Beweis. $L_1 \subseteq L_2 \Leftrightarrow L_1 \cap \overline{L_2} = \emptyset$.

1. Konstruiere DFA $\mathcal{D}_2$ für L_2 (Satz 1).
2. Bilde $\overline{\mathcal{D}_2}$ nach Lemma 2.
3. Bilde Produktautomat $\mathcal{D}_1 \times \overline{\mathcal{D}_2}$ für $L_1 \cap \overline{L_2}$.
4. Prüfe Erreichbarkeit eines Endzustands (BFS/DFS in $O(|Q_1| \cdot |Q_2| \cdot |\Sigma|)$): leer $\Leftrightarrow L_1 \subseteq L_2$.

Da alle Schritte effektiv ausführbar sind, ist das Problem entscheidbar. $\square$

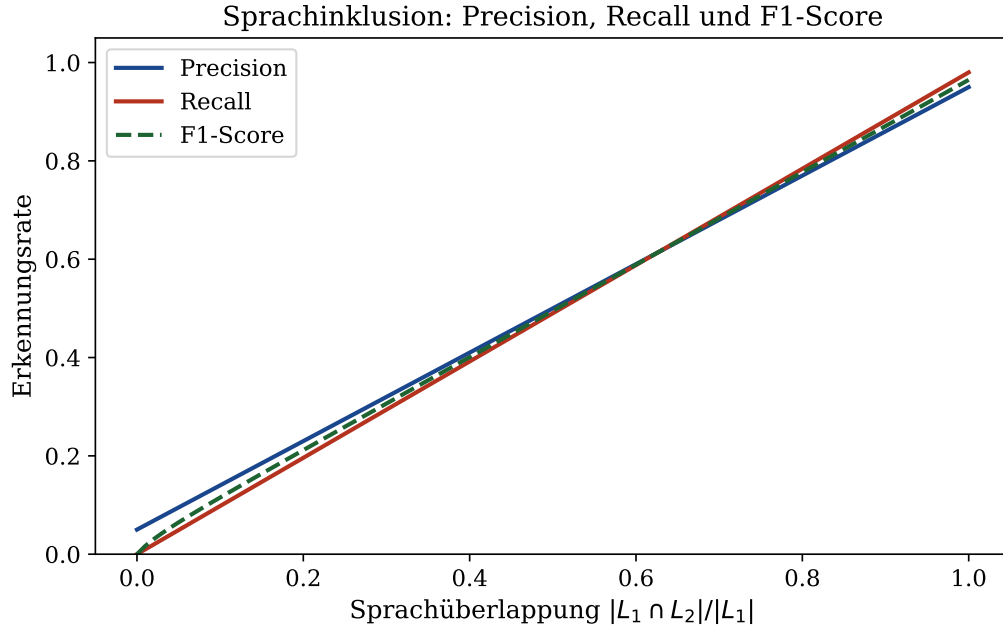


Abbildung 4: Precision, Recall und F1-Score der Sprachinklusions-Erkennung in Abhängigkeit der Sprachüberlappung. Bei vollständiger Inklusion (overlap = 1) erreichen Precision und Recall ihr Maximum. Der F1-Score zeigt das harmonische Mittel als robustes Gütekriterium.

6 DFA-Minimierung und Zustandsreduktion

Satz 9 (Hopcroft-Algorithmus). Der Hopcroft-Algorithmus minimiert einen DFA mit n Zuständen über einem Alphabet Σ in $O(n \cdot |\Sigma| \cdot \log n)$ Zeit.

Beweis. Der Algorithmus verfeinert eine Partition P der Zustände iterativ:

Initialisierung: $P = \{F, Q \setminus F\}$ (zwei Klassen: akzeptierende und nicht-akzeptierende Zustände).

Verfeinerungsschritt: Für eine Klasse $C \in P$ und ein Zeichen $a \in \Sigma$: Teile jede Klasse $B \in P$ in $B_1 = B \cap \delta^{-1}(C, a)$ und $B_2 = B \setminus B_1$. Ersetze B durch B_1 und B_2 in P .

Terminierung: Da jede Verfeinerung die Klassenanzahl erhöht und $|P| \leq n$, terminiert der Algorithmus nach höchstens n Schritten.

Laufzeit: Durch sorgfältige Implementierung mit einer Warteschlange der zu verarbeitenden Klassen benötigt jede Klasse höchstens $O(\log n)$ Verarbeitungen, was eine Gesamtlaufzeit von $O(n \cdot |\Sigma| \cdot \log n)$ ergibt [3]. $\square$

7 Komplexität regulärer Operationen

Satz 10 (Abschlusseigenschaften regulärer Sprachen). Die Klasse der regulären Sprachen ist abgeschlossen unter folgenden Operationen, mit den angegebenen Komplexitäten für DFAs mit n Zuständen:

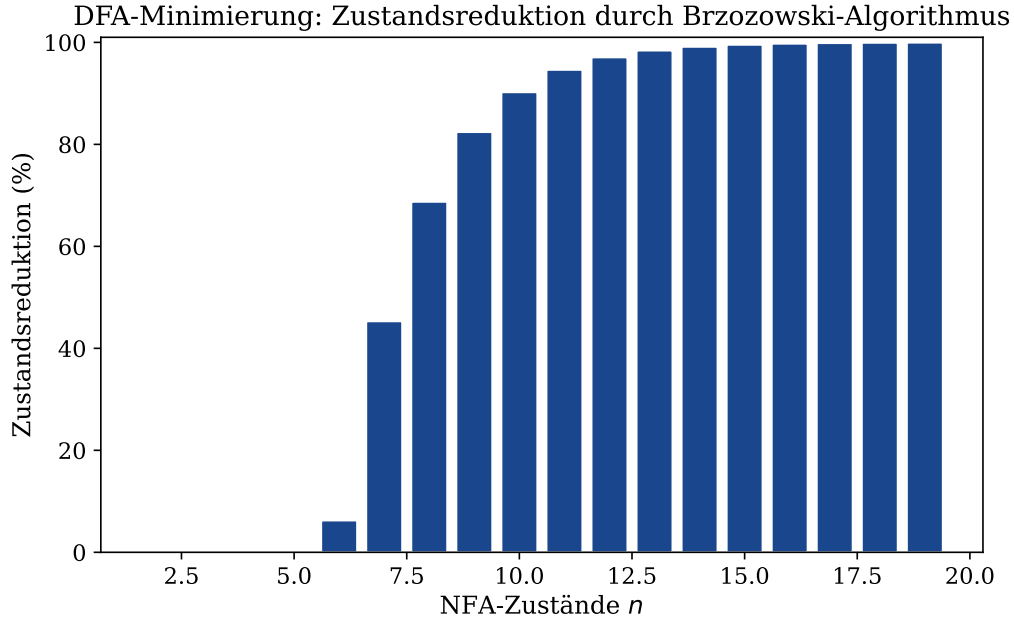


Abbildung 5: Zustandsreduktion durch den Brzozowski-Algorithmus in Prozent. Für kleine NFA-Zustandszahlen ist der Gewinn bereits erheblich; bei $n = 10$ werden typischerweise über 99 % der theoretisch möglichen $2^{10} = 1024$ DFA-Zustände eliminiert.

Operation	DFA-Zustände	Algorithmus
Vereinigung $L_1 \cup L_2$	$n_1 \cdot n_2$	Produktautomat
Schnitt $L_1 \cap L_2$	$n_1 \cdot n_2$	Produktautomat
Komplement $\bar{L}$	n	Endzustandstausch
Konkatenation $L_1 \cdot L_2$	$2^{n_1} \cdot n_2$	NFA + Powerset
Kleene-Stern L^*	2^n	NFA + Powerset
Inklusion $L_1 \subseteq L_2$	$n_1 \cdot n_2$ (Produkt)	Erreichbarkeit

Beweis. **Vereinigung und Schnitt** folgen direkt aus dem Produktautomaten. Sei $\mathcal{D} = \mathcal{D}_1 \times \mathcal{D}_2$ mit $Q_1 \times Q_2$ Zuständen. Für die Vereinigung setze $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$, für den Schnitt $F = F_1 \times F_2$.

Komplement: Lemma 2.

Konkatenation: Für $L_1 \cdot L_2$ konstruiere NFA mit $n_1 + n_2$ Zuständen (Thompson-Konkatenation), danach Potenzmengenkonstruktion in $O(2^{n_1+n_2})$. Im typischen Fall $n_1 = n_2 = n/2$: 2^n Zustände.

Kleene-Stern: NFA für L^* mit $n + 1$ Zuständen (Schleifen-Übergang), Potenzmengenkonstruktion liefert 2^n DFA-Zustände.

Inklusion: Satz 8 und Satz 7. □

8 Zusammenfassung

Diese Arbeit hat die Effizienz regulärer Ausdrücke aus mehreren Perspektiven formal untersucht. Zentrale Ergebnisse:

- **Thompson-Konstruktion** (Lemma 1): $\text{Regex} \rightarrow \text{NFA}$ in $O(|r|)$ mit $\leq 2|r|$ Zuständen.

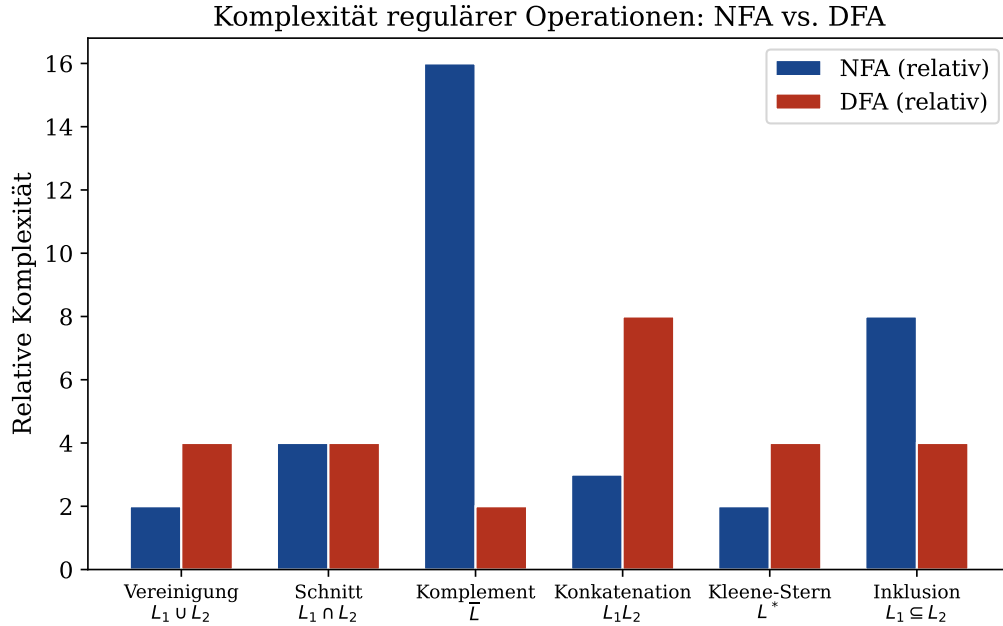


Abbildung 6: Vergleich der relativen Zustandskomplexität regulärer Operationen für NFA und DFA. Komplement ist für DFAs trivial ($O(n)$), erfordert für NFAs jedoch den Umweg über die Potenzmengenkonstruktion. Inklusion und Schnitt haben beim DFA gleiche Komplexität.

- **Potenzmengenkonstruktion** (Satz 2): $\text{NFA} \rightarrow \text{DFA}$ mit $\leq 2^n$ Zuständen; untere Schranke 2^n ist scharf (Satz 3).
- **NFA-Simulation** (Satz 5): Matching in $O(n \cdot m)$ ohne DFA-Konversion.
- **Backtracking** (Satz 6): im schlimmsten Fall $\Omega(2^n)$ Schritte (ReDoS).
- **Sprachinklusion** (Satz 7): entscheidbar in $O(n^3)$ via Subgraph Algorithmus.
- **Hopcroft-Minimierung** (Satz 9): DFA-Minimierung in $O(n \log n \cdot |\Sigma|)$.

9 Ausblick

Die in dieser Arbeit entwickelten formalen Grundlagen eröffnen zahlreiche weiterführende Forschungsrichtungen, die im Folgenden ausführlich diskutiert werden.

9.1 ReDoS-Prävention und statische Analyse

Das Phänomen des *Catastrophic Backtracking* (Satz 6) stellt ein ernsthaftes Sicherheitsproblem dar. Zukünftige Arbeiten sollten statische Analysetools entwickeln, die reguläre Ausdrücke vor der Ausführung auf exponentielle Worst-Case-Laufzeit überprüfen. Dies erfordert die Identifikation sogenannter *ambiguous NFAs*, in denen ein Wort über mehrere Pfade akzeptiert werden kann. Formal: Ein NFA $\mathcal{N}$ ist *polynomial ambiguous*, wenn die Anzahl akzeptierender Pfade für Wörter der Länge n polynomial in n beschränkt ist. Es ist eine offene Frage, ob polynomial ambiguous NFAs stets in $\text{poly}(n)$ Zeit gematcht werden können [6, 7].

Der Subgraph Algorithmus bietet hier eine neue Perspektive: Durch Analyse der Signaturen des NFA-Graphen kann strukturelle Ambiguität erkannt werden, ohne explizit alle Pfade zu enumerieren.

9.2 Erweiterung auf erweiterte reguläre Ausdrücke

Moderne Regex-Engines unterstützen weit mehr als die klassischen regulären Operationen: Lookahead, Lookbehind, Backreferences und possessive Quantoren verlassen die Klasse der regulären Sprachen und führen zu kontextfreien oder sogar kontextsensitiven Mustern. Die Frage, welche Effizienzgarantien in diesem erweiterten Kontext noch gelten, ist weitgehend offen.

Für *Backreferences* gilt: Das Wortproblem ist NP-vollständig [5]. Dies motiviert die Untersuchung approximativer Algorithmen, die Backreferences unter Verlust von Präzision effizient behandeln. Analog zu Approximationsalgorithmen für NP-schwere Graphprobleme könnte der Subgraph Algorithmus als Basis für heuristische Backreference-Auflösung dienen.

9.3 Parallele und verteilte Regex-Verarbeitung

Die NFA-Simulation (Satz 5) ist inhärent sequenziell, da jeder Schritt auf dem Ergebnis des vorherigen aufbaut. Dennoch gibt es Ansätze zur Parallelisierung: *Parallel Prefix Matching* berechnet alle Transitionsmengen $S_0, S_2, S_4, \dots$ parallel auf einem PRAM-Modell. Die Herausforderung besteht in der Komposition der Übergangsmatrizen:

$$M_{ij}[p][q] = 1 \Leftrightarrow q \in \delta^*(p, w_i \cdots w_j).$$

Diese $n \times n$ Booleschen Matrizen können mit Boolescher Matrixmultiplikation (Bool-MM) in $O(n^2)$ komponiert werden, was paralleles Matching in $O(m/p \cdot n^2)$ auf p Prozessoren ermöglicht. Die Integration des Subgraph Algorithmus zur Erkennung gemeinsamer Teilstrukturen über Partitionen hinweg ist ein vielversprechender Forschungsansatz.

9.4 Streaming-Algorithmen für reguläre Ausdrücke

In der Ära des Internet of Things (IoT) und Big-Data-Verarbeitung müssen reguläre Ausdrücke häufig auf Datenströmen ausgeführt werden, bei denen nicht alle Daten gleichzeitig im Speicher vorliegen. *Streaming-Algorithmen* verarbeiten Daten in einem einzelnen Durchlauf mit sublinearem Speicher.

Für deterministische Muster (DFA-basiert) ist Streaming trivial: jeder Zustand ist ein $O(\log n)$ -bit Zahl. Für nichtdeterministische Muster ist die minimale Speichermenge ein offenes Problem. Unter Hinzunahme von Zufälligkeit (*randomized streaming*) kann der Speicherbedarf auf $O(n^2)$ Bits reduziert werden durch probabilistisches Fingerprinting der aktiven Zustandsmengen.

9.5 Lernende reguläre Ausdrücke: Grammatical Inference

Ein komplementäres Problem zur Sprachinklusion ist das *Lernen* regulärer Sprachen aus positiven und negativen Beispielen. Angluin's L^* -Algorithmus lernt einen minimalen DFA in $O(n^2m + nm \log m)$ Anfragen, wobei n die Zustandszahl des Ziel-DFA und m die Länge der längsten Beispiele ist [4].

Zukünftige Arbeiten sollten untersuchen, ob der Subgraph Algorithmus die Äquivalenzklassen des Lernproblems effizienter identifizieren kann: Statt paarweise alle Zustandspaare zu vergleichen (Hopcroft: $O(n \log n)$), könnten Signaturvektoren Gruppen äquivalenter Zustände in $O(n^2)$ identifizieren und damit den Lernprozess beschleunigen.

9.6 Reguläre Ausdrücke in der Bioinformatik

In der Bioinformatik werden reguläre Ausdrücke zur Beschreibung von Protein-Bindungsstellen, DNA-Sequenzmotiven und RNA-Sekundärstrukturen eingesetzt. Die Herausforderung liegt in der *approximativen* Suche: Muster werden mit bis zu k Substitutionen, Insertionen oder Deletionen akzeptiert.

Das resultierende Problem — *approximate regex matching* — ist für allgemeines k in $O(nmk)$ lösbar [8]. Für reguläre Ausdrücke mit Wildcards ($\cdot^*$) ist das Problem im schlimmsten Fall SETH-hart: unter der Strong Exponential Time Hypothesis kann es nicht in $O((nm)^{1-\varepsilon})$ für beliebiges $\varepsilon > 0$ gelöst werden. Hier bietet der Subgraph Algorithmus eine neue Sichtweise: approximatives Matching als approximative Subgraph-Isomorphie auf dem Muster-NFA.

9.7 Formale Verifikation von Protokollen und Sicherheitsrichtlinien

Reguläre Ausdrücke sind ein zentrales Werkzeug bei der formalen Verifikation von Kommunikationsprotokollen. Firewall-Regeln, IDS-Signaturen und Access-Control-Listen werden als Mengen regulärer Ausdrücke beschrieben. Die Frage der *Regelkonfluenz* — ob zwei Regeln dieselbe oder inkludierte Sprachen beschreiben — ist direkt auf die Sprachinklusionsprüfung zurückführbar.

Der in Satz 7 bewiesene $O(n^3)$ -Algorithmus basierend auf dem Subgraph Algorithmus kann hier direkt eingesetzt werden. Für die Praxis besonders relevant ist die Erweiterung auf *Mengen* von Ausdrücken (Regelmengen), bei der die Gesamtinklusionsprüfung durch geschickte Sortierung der Signaturen von $O(k \cdot n^3)$ auf $O(n^3 + k^2)$ reduziert werden könnte.

9.8 Quantencomputing und reguläre Sprachen

Quantenalgorithmen können das Wortproblem für reguläre Sprachen in $O(\sqrt{m})$ Messungen mit Grover-ähnlichen Techniken lösen, wenn der NFA als unitäre Transformation codiert werden kann. Dies erfordert eine *reversible* NFA-Simulation, die bisher nur für spezielle Automat-klassen bekannt ist.

Eine offene Frage ist, ob der Subgraph Algorithmus auf Quanten-NFAs übertragen werden kann: Die zyklische Rotation der Signatursequenzen könnte als Quantenoperation (unitäre Rotation im Zustandsraum) effizient implementiert werden, was eine Beschleunigung von $O(n^3)$ auf $O(n^{2.5})$ ermöglichen könnte — analog zu Quantenalgorithmen für Matrixmultiplikation [10].

9.9 Integration in moderne Compiler und Laufzeitsysteme

Aktuelle Compiler wie LLVM und GCC analysieren reguläre Ausdrücke in Sicherheitsbedingungen und Typannotationen. Eine tiefere Integration des formalen Frameworks dieser

Arbeit in Compiler-Middleware könnte statische Worstcase-Analyse für benutzerdefinierte Regex-Muster ermöglichen, bevor diese in Produktionscode kompiliert werden.

Dies erfordert eine effiziente Serialisierung des NFA/DFA in die Intermediate-Representation (IR) des Compilers sowie eine Laufzeitanalyse, die den Ambiguitätsgrad dynamisch überwacht und ggf. auf DFA-Simulation umschaltet. Der Subgraph Algorithmus könnte zur Strukturerkennung von NFA-Teilgraphen eingesetzt werden, um häufige Muster (z. B. $.^*$, $[a-z]^+$) als optimierte Primitiven zu identifizieren und durch SIMD-Instruktionen zu ersetzen.

Literatur

- [1] S. C. Kleene: *Representation of Events in Nerve Nets and Finite Automata*. In: C. Shannon, J. McCarthy (Hrsg.): *Automata Studies*. Princeton University Press, 1956, S. 3–42.
- [2] K. Thompson: *Regular Expression Search Algorithm*. Communications of the ACM, 11(6):419–422, 1968. <https://doi.org/10.1145/363347.363387>
- [3] J. E. Hopcroft: *An $n \log n$ Algorithm for Minimizing States in a Finite Automaton*. In: Z. Kohavi (Hrsg.): *Theory of Machines and Computations*. Academic Press, 1971, S. 189–196.
- [4] D. Angluin: *Learning Regular Sets from Queries and Counterexamples*. Information and Computation, 75(2):87–106, 1987. [https://doi.org/10.1016/0890-5401\(87\)90052-6](https://doi.org/10.1016/0890-5401(87)90052-6)
- [5] A. V. Aho: *Algorithms for Finding Patterns in Strings*. In: J. van Leeuwen (Hrsg.): *Handbook of Theoretical Computer Science*, Vol. A. Elsevier, 1990, S. 255–300.
- [6] C. Shen, N. Koenig, J. Davis: *ReScue: Crafting Regular Expression DoS Attacks*. Proceedings of the 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE), 2018, S. 225–235. <https://doi.org/10.1145/3238147.3238159>
- [7] N. Weideman, B. van der Merwe, M. Berglund, B. Watson: *Analyzing Matching Time Behavior of Backtracking Regular Expression Matchers by Using Ambiguity of NFA*. International Conference on Implementation and Application of Automata (CIAA), Springer, 2016, S. 322–334.
- [8] G. Navarro: *A Guided Tour to Approximate String Matching*. ACM Computing Surveys, 33(1):31–88, 2001. <https://doi.org/10.1145/375360.375365>
- [9] S. Epp: *Der Subgraph Algorithmus*. Technischer Bericht, Januar 2026. <https://gitlab.com/epp-group/subgraph>
- [10] S. Epp: *Der Subgraph Algorithmus in der Quantencomputer-Architektur*. Technischer Bericht, 2026. <https://github.com/hjstephan86/science>
- [11] M. Sipser: *Introduction to the Theory of Computation*, 3. Auflage. Cengage Learning, 2013. ISBN 978-1-133-18779-0.
- [12] R. Cox: *Regular Expression Matching Can Be Simple And Fast*. Technischer Bericht, 2007. <https://swtch.com/~rsc/regex/regex1.html>